🔶 **LLD — Question Code: Q332 — Product Inventory Analytics**
Title: Product Inventory Analytics
Difficulty: Easy → Medium
Language: Python 3.x (NumPy/Pandas allowed)

📌 **Overview**
A retail company maintains daily inventory logs for each product in its warehouses.
Each inventory log consists of:

- ProductID (string or int)
- Category (string)
- Date (string: "YYYY-MM-DD")
- StockLevel (integer — number of units available)

Your task is to implement analytics over these logs to determine:

- Monthly average stock level per product
- Flag low-inventory days
- Identify products that frequently go below threshold
- Category-wise summary of inventory conditions
- Create clean, structured DataFrames from raw logs

📦 **Class & Functions to Implement**
Create a class named:

```
class InventoryAnalyzer:
   # implement methods defined below
```

No constructor required. Use Pandas for all operations.

🔥 **Functional Requirements (LLD format)**
Each method below specifies:

- **Function declaration**
- **Purpose**
- **Input & Output**
- **Implementation flow**
- **Expected behavior**
- **Edge cases**

✅ **1) Create Inventory DataFrame**
Function Declaration

```
def create_inventory_df(self, data: list[list]) -> pd.DataFrame:
```

Purpose
Convert raw logs into a structured Pandas DataFrame.

Input Format
data is a list of lists, where each element is:
[ProductID, Category, Date, StockLevel]

- ProductID: string or int
- Category: string

- Date: string "YYYY-MM-DD" (DO NOT convert to datetime)
- StockLevel: int

Output Format
A Pandas DataFrame with columns (in this order):
["ProductID", "Category", "Date", "StockLevel"]
The DataFrame must preserve row order.

Example
Input:
[
 ["P101", "Electronics", "2024-07-01", 50],
 ["P102", "Clothing", "2024-07-01", 20]
]
Output:
 ProductID Category Date StockLevel
0 P101 Electronics 2024-07-01 50
1 P102 Clothing 2024-07-01 20

Implementation Flow
- Define column names.
- Create a DataFrame using pd.DataFrame(data, columns=...).
- Do **not** parse or modify the Date column.

✅ **2) Compute Monthly Average Stock Per Product**
Function Declaration
def compute_monthly_avg_stock(self, df: pd.DataFrame) -> pd.DataFrame:

Purpose
Return the average stock level for each (ProductID, Month).

Input
df generated using create_inventory_df.

Output Format
Pandas DataFrame with columns:
["ProductID", "Month", "AverageStock"]
Where:
- Month = first 7 characters of Date → "YYYY-MM"
- AverageStock is the mean stock level (float)

Implementation Flow
1. Create new column:
2. df["Month"] = df["Date"].str[:7]
3. Group by ProductID and Month.
4. Aggregate:
    - Average stock using .mean().

5. Return sorted DataFrame by ProductID, Month.

Example

Input rows for July 2024:

ProductID	Date	StockLevel
P101	2024-07-01	50
P101	2024-07-02	70

Output:

	ProductID	Month	AverageStock
0	P101	2024-07	60.0

✅ **3) Add Low-Stock Flag Column**

Function Declaration

```
def add_low_stock_flag(self, df: pd.DataFrame, threshold: int) -> pd.DataFrame:
```

Purpose

Add a flag indicating if stock is **below a threshold**.

Input

- df: DataFrame from step 1
- threshold: integer (stock threshold value)

Output

Adds a new column:
IsLowStock = 1 if StockLevel < threshold else 0

Implementation Flow

- Use vectorized comparison:
- df["IsLowStock"] = (df["StockLevel"] < threshold).astype(int)
- Return the modified DataFrame (copy or mutated).

Example

threshold = 30
Input:

StockLevel
50
20

Output:

StockLevel	IsLowStock
50	0
20	1

✅ **4) Identify Frequently Low-Stock Products**

Function Declaration

```
def frequent_low_stock(self, df: pd.DataFrame, threshold: int) -> pd.DataFrame:
```

Purpose

Identify products whose number of **low-stock days** exceeds the threshold.

Input

- df: DataFrame with "IsLowStock" column.
- threshold: int -> only include products with LowStockCount > threshold.

Output Format

["ProductID", "LowStockCount"]

Sorted:

- by LowStockCount DESC
- if tie → by ProductID ASC

Implementation Flow

1. Filter df[df["IsLowStock"] == 1].
2. Group by ProductID and count low-stock rows.
3. Filter where count > threshold.
4. Sort.
5. Reset index.

Example

Product low-stock counts:

- P101 → 1
- P102 → 3

threshold = 2

Output:

	ProductID	LowStockCount
0	P102	3

✅ **5) Category-wise Stock Summary**

Function Declaration

def category_stock_summary(self, df: pd.DataFrame) -> pd.DataFrame:

Purpose

Create a summary of stock conditions per category.

It must compute counts of:

- "Present" → StockLevel > 0
- "OutOfStock" → StockLevel == 0
- "LowStock" → StockLevel < 10

But this classification is internal — no new Status column needed.

Output Format

DataFrame with columns:

["Category", "Present", "OutOfStock", "LowStock"]

Implementation Flow

1. Derive temporary status column:
2. if StockLevel == 0 → "OutOfStock"
3. elif StockLevel < 10 → "LowStock"
4. else → "Present"
5. Generate a crosstab:
6. pd.crosstab(df["Category"], temp_status)
7. Ensure column order: Present, OutOfStock, LowStock
8. Fill missing columns with 0.
9. Reset index.

Example

Input:

Category	StockLevel
Electronics	0
Electronics	5
Clothing	20

Output:

	Category	Present	OutOfStock	LowStock
0	Electronics	0	1	1
1	Clothing	1	0	0

🔍 **Edge Cases to Handle**

Case 1 — Empty Input

Return empty DataFrames with **correct schema**.

Case 2 — Products with only 1 record

Monthly average should still compute correctly.

Case 3 — No low-stock days

frequent_low_stock must return an empty DataFrame.

Case 4 — Category does not have all three status types

Fill missing counts with 0.

Case 5 — StockLevel negative

Treat <0 as "LowStock" (real-world: invalid but safe handling).

📦 **Complexity Analysis**

Method	Time Complexity	Space Complexity
create_inventory_df	O(N)	O(N)
compute_monthly_avg_stock	O(N)	O(U)
add_low_stock_flag	O(N)	O(N)
frequent_low_stock	O(N)	O(P)
category_stock_summary	O(N)	O(C)

Where:

U = unique (ProductID, Month) groups

P = unique products

C = categories

🧪 **Sample Test Cases (convertible to pytest)**

Test 1 — Create DF

Assert columns equal to:

["ProductID","Category","Date","StockLevel"]

Test 2 — Monthly Average Computation

Input rows for PID P101:

Date	Stock
2024-07-01	50
2024-07-02	70

Expected:

P101 2024-07 60.0

Test 3 — Low-stock flag

threshold = 25

StockLevel=[10,50] → IsLowStock=[1,0]

Test 4 — Frequent low stock

Threshold = 2

Counts:

- P101 → 1
- P102 → 3

Expected output: P102 only.

Test 5 — Category summary

Given:

Cat	Level
A	0
A	5
B	12

Expected:

Category	Present	OutOfStock	LowStock
A	0	1	1
B	1	0	0

```
# inventory_analyzer.py
"""
InventoryAnalyzer - Implements inventory analytics as per LLD Q332.

Methods:
- create_inventory_df(data)
- compute_monthly_avg_stock(df)
- add_low_stock_flag(df, threshold)
- frequent_low_stock(df, threshold)
- category_stock_summary(df)

Author: Generated for Q332
"""

from typing import List
import pandas as pd
import numpy as np


class InventoryAnalyzer:
    def create_inventory_df(self, data: List[List]) -> pd.DataFrame:
        cols = ["ProductID", "Category", "Date", "StockLevel"]
        if data is None:
            return pd.DataFrame(columns=cols)
     df = pd.DataFrame(data, columns=cols)
     df['StockLevel'] = pd.to_numeric(df['StockLevel'], errors='coerce')
    return df

  def compute_monthly_avg_stock(self, df: pd.DataFrame) -> pd.DataFrame:
      expected_cols = {"ProductID", "Category", "Date", "StockLevel"}
    if df is None or df.empty:
      return pd.DataFrame(columns=["ProductID", "Month", "AverageStock"])
    # Work on a copy
    d = df.copy()
    if 'Date' not in d.columns:
      raise KeyError("Input DataFrame must contain 'Date' column.")
    # Month extraction (first 7 chars)
    d['Month'] = d['Date'].astype(str).str[:7]
    # Ensure StockLevel numeric; rows with NaN will be excluded from mean by default
    d['StockLevel'] = pd.to_numeric(d['StockLevel'], errors='coerce')
        # Group and compute mean
    grp = d.groupby(['ProductID', 'Month'], as_index=False)['StockLevel'].mean()
    grp = grp.rename(columns={'StockLevel': 'AverageStock'})

    # Sort and reset index
    grp = grp.sort_values(['ProductID', 'Month']).reset_index(drop=True)
    return grp[['ProductID', 'Month', 'AverageStock']]
```

```
    def add_low_stock_flag(self, df: pd.DataFrame, threshold: int) -> pd.DataFrame:
        """
        Add 'IsLowStock' column: 1 if StockLevel < threshold else 0.

        Parameters
        ----------
        df : pd.DataFrame
            Input must have 'StockLevel' column.
        threshold : int
            Threshold below which stock is considered low.

        Returns
        -------
        pd.DataFrame
            New DataFrame (copy) with added 'IsLowStock' column of ints (0/1).
        """
        if df is None or df.empty:
            return pd.DataFrame(columns=(list(df.columns) + ['IsLowStock']) if df is not None else
["ProductID", "Category", "Date", "StockLevel", "IsLowStock"])

        if 'StockLevel' not in df.columns:
            raise KeyError("Input DataFrame must contain 'StockLevel' column.")

        d = df.copy()
        # coercion ensures comparison works; NaN treated as False (not low) by default
        d['StockLevel'] = pd.to_numeric(d['StockLevel'], errors='coerce')

        # Mark negative stock as low as well (per LLD)
        d['IsLowStock'] = ((d['StockLevel'] < threshold) | (d['StockLevel'].lt(0))).astype(int)
        return d

    def frequent_low_stock(self, df: pd.DataFrame, threshold: int) -> pd.DataFrame:
        """
        Identify products with LowStockCount > threshold.

        Input df is expected to have column 'IsLowStock'. If not present,
        this method will compute low-stock using default cutoff < 10 (as per LLD)
        before counting.

        Parameters
        ----------
        df : pd.DataFrame
        threshold : int
            Minimum number of low-stock days (strictly greater) to include a product.

        Returns
        -------
        pd.DataFrame
```

```
        Columns: ['ProductID', 'LowStockCount'] sorted by LowStockCount DESC, ProductID ASC
        """
        if df is None or df.empty:
            return pd.DataFrame(columns=['ProductID', 'LowStockCount'])

        d = df.copy()

        # If IsLowStock not present, derive using default < 10 (LLD specified <10 for category LowStock)
        if 'IsLowStock' not in d.columns:
            # compute using StockLevel < 10; treat negative as low as well
            if 'StockLevel' not in d.columns:
                raise KeyError("Input DataFrame must contain 'IsLowStock' or 'StockLevel' column.")
            d['StockLevel'] = pd.to_numeric(d['StockLevel'], errors='coerce')
            d['IsLowStock'] = ((d['StockLevel'] < 10) | (d['StockLevel'].lt(0))).astype(int)

        # Filter low stock rows
        low = d[d['IsLowStock'] == 1]

        if low.empty:
            return pd.DataFrame(columns=['ProductID', 'LowStockCount'])

        counts = low.groupby('ProductID').size().reset_index(name='LowStockCount')

        # filter > threshold
        res = counts[counts['LowStockCount'] > threshold].copy()

        if res.empty:
            return pd.DataFrame(columns=['ProductID', 'LowStockCount'])

        # sort by LowStockCount desc, ProductID asc (ProductID may be numeric or string)
        res = res.sort_values(by=['LowStockCount', 'ProductID'], ascending=[False,
True]).reset_index(drop=True)
        return res[['ProductID', 'LowStockCount']]

    def category_stock_summary(self, df: pd.DataFrame) -> pd.DataFrame:
        """
        Produce category-wise counts of Present, OutOfStock, LowStock.

        Definitions:
            - OutOfStock: StockLevel == 0
            - LowStock: StockLevel < 10  (and StockLevel > 0), negative treated as LowStock
            - Present: StockLevel >= 10

        Returns DataFrame columns: ['Category','Present','OutOfStock','LowStock']
        """
        # Desired final columns (order)
        cols_order = ['Present', 'OutOfStock', 'LowStock']
```

```
    if df is None or df.empty:
        return pd.DataFrame(columns=['Category'] + cols_order)

    if 'Category' not in df.columns or 'StockLevel' not in df.columns:
        raise KeyError("Input DataFrame must contain 'Category' and 'StockLevel' columns.")

    d = df.copy()
    d['StockLevel'] = pd.to_numeric(d['StockLevel'], errors='coerce').fillna(0)

    # Build temporary status column using vectorized np.select
    # Order matters: OutOfStock (==0) first, then LowStock (<10), else Present
    cond_out = d['StockLevel'] == 0
    cond_low = (d['StockLevel'] < 10) & (d['StockLevel'] != 0)
    cond_present = d['StockLevel'] >= 10

    choices = ['OutOfStock', 'LowStock', 'Present']
    conditions = [cond_out, cond_low, cond_present]

    # We will map to desired labels, but np.select needs choices aligned
    # Instead, create a status series:
    status = pd.Series(np.select(conditions, choices, default='Present'), index=d.index)

    # Create crosstab
    ctab = pd.crosstab(d['Category'], status)

    # Ensure necessary columns present in desired order 'Present','OutOfStock','LowStock'
    ctab = ctab.reindex(columns=cols_order, fill_value=0)

    # Reset index to have Category as column
    result = ctab.reset_index()

    # Ensure integer types
    for c in cols_order:
        if c in result.columns:
            result[c] = result[c].astype(int)

    return result[['Category'] + cols_order]
```

Notes & usage tips

- All methods return pd.DataFrame.
- create_inventory_df converts StockLevel to numeric (NaN if parsing fails). You can adjust behavior if you wish to treat parsing errors differently.
- frequent_low_stock will compute a fallback IsLowStock using < 10 if the column isn't already present (this follows the LLD internal LowStock definition). If you'd rather require add_low_stock_flag() to be run explicitly first, remove the fallback and raise an error instead.
- Empty input DataFrames yield empty outputs with the correct schema.
- The implementation is vectorized and suitable for reasonably large CSVs; for very large data use chunked processing with pd.read_csv(chunksize=...).

LLD — Question Code: Q331 — *Employee Leave Analysis* (Low-Level Design)
Title: Employee Leave Analysis
Difficulty: Easy → Medium
Language: Python 3.x (Pandas allowed)

Overview
An organization stores daily attendance logs for employees. Each log entry includes:

- EmployeeID (int)
- Team (string) — team/department name
- Date (string in YYYY-MM-DD format)
- Status (string) — one of {"Present", "Absent", "Leave", "WorkFromHome"}

Implement LeaveAnalyzer class with multiple operations that transform and analyze the attendance logs.

File / Class requirement
Create a Python module with the class:

```
class LeaveAnalyzer:
  # No constructor required; you may implement __init__ if needed
  ...
```

All functions are instance methods of LeaveAnalyzer. Use standard Pandas for DataFrame operations.

API (Function declarations, types & behavior)
1) create_leave_df(self, data: list[list]) -> pd.DataFrame
Description: Build a Pandas DataFrame from raw rows.
Input:

- data: List[List] — each element is a row [EmployeeID, Team, Date, Status]
    - EmployeeID: integer (or string-encoded integer)
    - Team: string

- Date: string in "YYYY-MM-DD" format **(must be kept as string)** — **do not** convert to datetime.
- Status: string; expected values include "Present", "Absent", "Leave", "WorkFromHome"

Output:

- pd.DataFrame with columns (in **this order**): ["EmployeeID", "Team", "Date", "Status"]

Notes / Implementation details:

- Preserve input order.
- Do not modify types other than what DataFrame naturally infers.
- Ensure column names exactly match casing above.

Example:

input:

```
[
 [101, "Sales", "2024-06-01", "Present"],
 [102, "HR",    "2024-06-01", "Absent"]
]
```

output DataFrame:

	EmployeeID	Team	Date	Status
0	101	Sales	2024-06-01	Present
1	102	HR	2024-06-01	Absent

---

2) compute_monthly_leave_rate(self, df: pd.DataFrame) -> pd.DataFrame

Description: For each employee and month (derived from Date), compute the percentage of days where Status == "Leave".

Input:

- df: pd.DataFrame created by create_leave_df (or having same schema). Date is a string "YYYY-MM-DD".

Output:

- pd.DataFrame with columns (exact order): ["EmployeeID", "Month", "Leave Rate"]
  - EmployeeID: same type as in input
  - Month: string "YYYY-MM" (first 7 characters of Date)
  - Leave Rate: float representing percentage in range [0.0, 100.0] (no rounding requirement unless specified by tests; but recommended to keep at least one decimal — however testers will compare numerically)

Computation steps (recommended):

1. Create a copy (to avoid mutating df) and compute Month = df["Date"].str[:7].
2. Compute total records per (EmployeeID, Month) => TotalDays.
   - Use groupby(['EmployeeID','Month']).size() or .agg(size).
3. Compute leave records per (EmployeeID, Month) where Status == "Leave" => LeaveDays.
4. Merge totals and leave counts (left-join totals with leave counts), filling missing LeaveDays with 0.
5. Compute Leave Rate = (LeaveDays / TotalDays) * 100.
6. Return DataFrame sorted by EmployeeID, then Month (ascending) and reset index.

Edge Cases:

- If an employee-month has no Leave rows, Leave Rate should be 0.0.

- If a month has zero total days (shouldn't happen for proper input), avoid division-by-zero; you may omit such rows or set Leave Rate to 0.0. (Tests will not include zero-day groups.)
- Preserve EmployeeID type (if it's an int in input, keep as int in output).

Example:

input rows for June 2024:

```
[ [201, "Sales", "2024-06-01", "Present"],
  [201, "Sales", "2024-06-02", "Leave"],
  [202, "HR",    "2024-06-01", "Absent"] ]
```

output:

```
 EmployeeID  Month     Leave Rate
0     201     2024-06     50.0   # 1 leave / 2 total
1     202     2024-06      0.0   # 0 leave / 1 total
```

Complexity: O(N) time where N = number of rows (dominant cost: single pass groupby/aggregation). Space O(U) where U = number of unique (EmployeeID, Month) groups.

---

3) add_onleave_flag(self, df: pd.DataFrame) -> pd.DataFrame

Description: Add a column IsOnLeave with value 1 if Status == "Leave", else 0.

Input:

- df: pd.DataFrame with schema ["EmployeeID","Team","Date","Status"].

Output:

- pd.DataFrame: same rows and columns plus a new column 'IsOnLeave' appended at the end (or present), with integer values 0 or 1.

Implementation notes:

- Do not mutate the original DataFrame passed in — return a new DataFrame copy (unless tests expect in-place; default safe approach: return new df).
- Use vectorized operation: (df['Status'] == 'Leave').astype(int)

Example:

input row: [101, 'Sales', '2024-06-01', 'Leave']

output row: [101, 'Sales', '2024-06-01', 'Leave', 1]

---

4) frequent_onleave_employees(self, df: pd.DataFrame, threshold: int) -> pd.DataFrame

Description: Identify employees who were on Leave strictly more than threshold times. Return counts per employee.

Input:

- df: pd.DataFrame
- threshold: int (>=0) — only employees with Leave Count > threshold must be returned.

Output:

- pd.DataFrame with columns: ["EmployeeID", "Leave Count"] sorted in descending order of Leave Count (tie-breaker: ascending EmployeeID), reset_index(drop=True).

Implementation notes:

1. Filter df to rows where Status == 'Leave'.
2. Group by EmployeeID and count (size()), name count column "Leave Count".
3. Filter counts to > threshold.
4. Sort by 'Leave Count' descending, then EmployeeID ascending.
5. Reset index and return.

Edge Cases:

- If no employee exceeds threshold, return empty DataFrame with the correct columns.

- If threshold is 0, return employees with at least 1 leave (> 0).

Example:
df has leaves: Employee 201 -> 2 leaves; Employee 202 -> 4 leaves
threshold = 1
returns:

EmployeeID	Leave Count
202	4
201	2

---

5) team_leave_summary(self, df: pd.DataFrame) -> pd.DataFrame
Description: For each Team, compute counts of rows for each Status category. Ensure columns are present in the exact order:
['Team', 'Present', 'Absent', 'Leave', 'WorkFromHome']. Fill missing statuses with 0.
Input:

- df: pd.DataFrame

Output:

- pd.DataFrame with one row per Team and columns:
  ['Team','Present','Absent','Leave','WorkFromHome']
  - All counts are integers (0 when no rows for that status)
  - Team is in the DataFrame index or as a column — tests expect it as a normal column (not index)

Implementation notes:

- Use pd.crosstab(df['Team'], df['Status']) or df.groupby(['Team','Status']).size().unstack(fill_value=0).
- After crosstab/unstack, reindex columns to the ordered list ['Present','Absent','Leave','WorkFromHome'] with fill_value=0.
- Reset index to make Team a column (so final DF columns are Team, Present, Absent, Leave, WorkFromHome).

Edge Cases:

- If a team lacks a status, the count must be 0 (use fill_value=0).
- If a status never exists in dataset, still include the column with zeros.

Example:
input rows:
[[101,'Sales','2024-06-01','Present'],
[102,'Sales','2024-06-01','Absent'],
[103,'Sales','2024-06-02','Leave'],
[201,'HR','2024-06-01','Absent']]

output:

	Team	Present	Absent	Leave	WorkFromHome
0	Sales	1	1	1	0
1	HR	0	1	0	0

Additional Requirements & Style

- Use **vectorized Pandas** operations; avoid row-by-row Python loops (.iterrows) for performance.
- Functions must be **deterministic** and **pure** (do not rely on global state). Each method should **not** change the caller's DataFrame unless specified.

- Keep Date as string (do not convert to datetime) except when you explicitly need to (the task forbids conversion; use str[:7] for Month extraction).
- Return types must be Pandas DataFrames as specified.
- Column names and order should exactly match the specification to pass strict tests.

Robustness & Edge Cases Summary

1. **Missing or unexpected statuses:** If Status contains values not in ['Present','Absent','Leave','WorkFromHome'], team_leave_summary must still include only the required four columns (ignore or drop other statuses). However, tests will usually supply only the expected statuses.
2. **Missing Date or malformed date strings:** compute_monthly_leave_rate assumes Date is YYYY-MM-DD. If malformed values exist, Date.str[:7] will still produce some string — you may use errors='coerce' if converting, but spec forbids conversion. Tests will use valid date strings.
3. **Empty inputs:** Each method should return an empty DataFrame with correct columns on empty input.
4. **Large datasets:** Methods should use vectorized operations to be efficient.

Sample Test Cases (you can convert these to pytest)

TestCase 1 — Small sanity test

Input rows:

```
[
 [101,'Sales','2024-06-01','Present'],
 [101,'Sales','2024-06-02','Leave'],
 [102,'HR','2024-06-01','Absent']
]
```

Expectations:

- create_leave_df returns DataFrame with 3 rows and columns EmployeeID, Team, Date, Status.
- compute_monthly_leave_rate returns:

EmployeeID	Month	Leave Rate
101	2024-06	50.0
102	2024-06	0.0

- add_onleave_flag adds IsOnLeave values 0,1,0.
- frequent_onleave_employees(df, threshold=0) returns EmployeeID 101 with Leave Count 1.
- team_leave_summary returns rows for Sales and HR with counts per status.

TestCase 2 — Multiple months

- Include rows in 2024-05 and 2024-06 to ensure Month groups work correctly.

TestCase 3 — No leaves

- All statuses are Present or WorkFromHome. compute_monthly_leave_rate should return Leave Rate = 0.0 for all employee-months. frequent_onleave_employees with any threshold >=0 returns empty DataFrame.

TestCase 4 — Empty input

- All methods return DataFrames with correct schema but zero rows.

Example harness (plain Python assertions)

```
from leave_analyzer import LeaveAnalyzer
import pandas as pd
```

```
an = LeaveAnalyzer()
data = [
 [301, "Eng", "2024-06-01", "Present"],
 [301, "Eng", "2024-06-02", "Leave"],
 [302, "HR",  "2024-06-02", "Leave"],
]

df = an.create_leave_df(data)
assert list(df.columns) == ["EmployeeID","Team","Date","Status"]
mr = an.compute_monthly_leave_rate(df)
# check EmployeeID 301 has 50% leave
assert float(mr[(mr.EmployeeID==301)&(mr.Month=='2024-06')]['Leave Rate']) == 50.0
df2 = an.add_onleave_flag(df)
assert df2['IsOnLeave'].tolist() == [0,1,1]
freq = an.frequent_onleave_employees(df, threshold=0)
assert 302 in freq['EmployeeID'].values
summary = an.team_leave_summary(df)
for col in ['Present','Absent','Leave','WorkFromHome']:
   assert col in summary.columns
```

Below are two ready-to-run files:

1. leave_analyzer.py — the complete implementation following the LLD (Q331).
2. test_leave_analyzer.py — a pytest test suite that implements the sample tests (including edge cases).

Save these files in the same folder. Create & activate a virtualenv, install pandas and pytest, then run: pytest -q

---

File: leave_analyzer.py

```
# leave_analyzer.py
"""
LeaveAnalyzer - Implements attendance/leave analytics as per LLD Q331.

Methods:
- create_leave_df(data)
- compute_monthly_leave_rate(df)
- add_onleave_flag(df)
- frequent_onleave_employees(df, threshold)
- team_leave_summary(df)

Author: Generated for Q331
"""

from typing import List
import pandas as pd
import numpy as np
```

```
class LeaveAnalyzer:
    """
    LeaveAnalyzer provides methods to analyze employee leave patterns.
    """

    def create_leave_df(self, data: List[List]) -> pd.DataFrame:
        """
        Create a DataFrame from raw attendance logs.

        Parameters
        ----------
        data : List[List]
            Each inner list must be: [EmployeeID, Department/Team, Date (YYYY-MM-DD str),
Attendance/status]

        Returns
        -------
        pd.DataFrame
            DataFrame with columns ["EmployeeID", "Department", "Date", "Attendance"]
            Preserves row order and keeps Date as string.
        """
        cols = ["EmployeeID", "Department", "Date", "Attendance"]
        # Handle None / empty gracefully
        if data is None:
            return pd.DataFrame(columns=cols)
        df = pd.DataFrame(data, columns=cols)
        return df

    def compute_monthly_leave_rate(self, df: pd.DataFrame) -> pd.DataFrame:
        """
        Compute percentage of 'Leave' days per employee per month.

        Month is derived by df['Date'].str[:7] -> 'YYYY-MM'

        Returns:
            DataFrame with columns ['EmployeeID', 'Month', 'Leave Rate']
        """
        if df is None:
            return pd.DataFrame(columns=['EmployeeID', 'Month', 'Leave Rate'])

        if df.empty:
            return pd.DataFrame(columns=['EmployeeID', 'Month', 'Leave Rate'])

        # Validate presence of Date and Attendance
        if 'Date' not in df.columns or 'Attendance' not in df.columns or 'EmployeeID' not in df.columns:
            raise KeyError("Input DataFrame must contain 'EmployeeID', 'Date', and 'Attendance'
columns.")
```

```
        d = df.copy()
        # Ensure Date treated as string; extract YYYY-MM
        d['Month'] = d['Date'].astype(str).str[:7]

        # Total days per (EmployeeID, Month)
        total = d.groupby(['EmployeeID', 'Month'], as_index=False).size().rename(columns={'size':
'TotalDays'})
        # Leave days per (EmployeeID, Month)
        leaves = d[d['Attendance'] == 'Leave'].groupby(['EmployeeID', 'Month'],
as_index=False).size().rename(columns={'size': 'LeaveDays'})

        merged = pd.merge(total, leaves, on=['EmployeeID', 'Month'], how='left')
        merged['LeaveDays'] = merged['LeaveDays'].fillna(0).astype(int)

        # Compute rate
        merged['Leave Rate'] = (merged['LeaveDays'] / merged['TotalDays']) * 100.0

        result = merged[['EmployeeID', 'Month', 'Leave Rate']].sort_values(['EmployeeID',
'Month']).reset_index(drop=True)
        return result

    def add_onleave_flag(self, df: pd.DataFrame) -> pd.DataFrame:
        """
        Add column 'IsOnLeave' with 1 if Attendance == 'Leave' else 0.

        Returns a new DataFrame (does not mutate input).
        """
        if df is None:
            return pd.DataFrame(columns=['EmployeeID', 'Department', 'Date', 'Attendance', 'IsOnLeave'])

        if df.empty:
            # maintain column list if present otherwise use default
            cols = list(df.columns) + ['IsOnLeave'] if df is not None else ['EmployeeID', 'Department', 'Date',
'Attendance', 'IsOnLeave']
            return pd.DataFrame(columns=cols)

        if 'Attendance' not in df.columns:
            raise KeyError("Input DataFrame must contain 'Attendance' column.")

        d = df.copy()
        d['IsOnLeave'] = (d['Attendance'] == 'Leave').astype(int)
        return d

    def frequent_onleave_employees(self, df: pd.DataFrame, threshold: int) -> pd.DataFrame:
        """
        Return employees with Leave Count > threshold.

        Output columns: ['EmployeeID', 'Leave Count'] sorted by Leave Count DESC, EmployeeID ASC.
```

```
        """
        if df is None:
            return pd.DataFrame(columns=['EmployeeID', 'Leave Count'])

        if df.empty:
            return pd.DataFrame(columns=['EmployeeID', 'Leave Count'])

        d = df.copy()

        # If Attendance column present, use it; else, expect IsOnLeave
        if 'Attendance' in d.columns:
            leaves = d[d['Attendance'] == 'Leave']
            if leaves.empty:
                return pd.DataFrame(columns=['EmployeeID', 'Leave Count'])
            counts = leaves.groupby('EmployeeID').size().reset_index(name='Leave Count')
        elif 'IsOnLeave' in d.columns:
            leaves = d[d['IsOnLeave'] == 1]
            if leaves.empty:
                return pd.DataFrame(columns=['EmployeeID', 'Leave Count'])
            counts = leaves.groupby('EmployeeID').size().reset_index(name='Leave Count')
        else:
            raise KeyError("Input DataFrame must contain 'Attendance' or 'IsOnLeave' column.")

        # Filter > threshold
        res = counts[counts['Leave Count'] > threshold].copy()
        if res.empty:
            return pd.DataFrame(columns=['EmployeeID', 'Leave Count'])

        res = res.sort_values(by=['Leave Count', 'EmployeeID'], ascending=[False,
True]).reset_index(drop=True)
        return res[['EmployeeID', 'Leave Count']]

    def team_leave_summary(self, df: pd.DataFrame) -> pd.DataFrame:
        """
        Returns counts of each attendance type per Department (team).

        Output columns: ['Department', 'Present', 'Absent', 'Leave']
        """
        desired_statuses = ['Present', 'Absent', 'Leave']

        if df is None:
            return pd.DataFrame(columns=['Department'] + desired_statuses)

        if df.empty:
            return pd.DataFrame(columns=['Department'] + desired_statuses)

        if 'Department' not in df.columns or 'Attendance' not in df.columns:
            raise KeyError("Input DataFrame must contain 'Department' and 'Attendance' columns.")
```

```
    d = df.copy()

    ctab = pd.crosstab(d['Department'], d['Attendance'])
    # Only include desired_statuses columns and fill missing with 0
    ctab = ctab.reindex(columns=desired_statuses, fill_value=0)
    result = ctab.reset_index()
    # ensure integer type
    for col in desired_statuses:
        if col in result.columns:
            result[col] = result[col].astype(int)
    return result[['Department'] + desired_statuses]
```

File: test_leave_analyzer.py

```
# test_leave_analyzer.py
import pytest
import pandas as pd
from leave_analyzer import LeaveAnalyzer

@pytest.fixture
def sample_data():
    # Small sample covering multiple behaviors
    data = [
        [301, "Engineering", "2024-06-01", "Present"],
        [302, "HR",          "2024-06-01", "Leave"],
        [301, "Engineering", "2024-06-02", "Leave"],
        [303, "Engineering", "2024-06-01", "Absent"],
        [302, "HR",          "2024-06-02", "Leave"],
        [302, "HR",          "2024-06-03", "Leave"],
        [301, "Engineering", "2024-06-03", "Present"],
        [304, "Sales",       "2024-06-01", "WorkFromHome"],
        [304, "Sales",       "2024-06-02", "Present"],
        [304, "Sales",       "2024-06-03", "Leave"],
    ]
    return data

def test_create_leave_df_and_columns(sample_data):
    an = LeaveAnalyzer()
    df = an.create_leave_df(sample_data)
    assert isinstance(df, pd.DataFrame)
    assert list(df.columns) == ["EmployeeID", "Department", "Date", "Attendance"]
    assert df.shape[0] == len(sample_data)

def test_add_onleave_flag(sample_data):
    an = LeaveAnalyzer()
    df = an.create_leave_df(sample_data)
    df_flagged = an.add_onleave_flag(df)
    # Count leaves in raw data
```

```
    expected_leaves = sum(1 for row in sample_data if row[3] == 'Leave')
    assert 'IsOnLeave' in df_flagged.columns
    assert df_flagged['IsOnLeave'].sum() == expected_leaves
    # Check positions match
    leaves_positions = [i for i, r in enumerate(sample_data) if r[3] == 'Leave']
    for pos in leaves_positions:
        assert df_flagged.iloc[pos]['IsOnLeave'] == 1

def test_compute_monthly_leave_rate(sample_data):
    an = LeaveAnalyzer()
    df = an.create_leave_df(sample_data)
    mr = an.compute_monthly_leave_rate(df)
    # There should be rows for EmployeeIDs 301,302,303,304 for 2024-06
    assert set(mr['EmployeeID'].unique()) == {301,302,303,304}
    # Check employee 302: has 3 leave days out of 3 => 100.0
    row_302 = mr[(mr['EmployeeID'] == 302) & (mr['Month'] == '2024-06')]
    assert not row_302.empty
    assert pytest.approx(float(row_302['Leave Rate'].iloc[0]), rel=1e-6) == 100.0
    # Check employee 301: 1 leave out of 3 => 33.333... approx
    row_301 = mr[(mr['EmployeeID'] == 301) & (mr['Month'] == '2024-06')]
    assert not row_301.empty
    assert pytest.approx(float(row_301['Leave Rate'].iloc[0]), rel=1e-3) == (1/3)*100

def test_frequent_onleave_employees(sample_data):
    an = LeaveAnalyzer()
    df = an.create_leave_df(sample_data)
    freq = an.frequent_onleave_employees(df, threshold=2)
    # only employee 302 has 3 leaves > 2
    assert freq.shape[0] == 1
    assert freq.iloc[0]['EmployeeID'] == 302
    assert freq.iloc[0]['Leave Count'] == 3

def test_team_leave_summary(sample_data):
    an = LeaveAnalyzer()
    df = an.create_leave_df(sample_data)
    summary = an.team_leave_summary(df)
    # Departments present: Engineering, HR, Sales
    assert set(summary['Department'].unique()) == {"Engineering", "HR", "Sales"}
    # ensure columns
    for col in ['Present','Absent','Leave']:
        assert col in summary.columns
    # check simple counts: HR should have 3 Leave entries
    hr_row = summary[summary['Department'] == 'HR'].iloc[0]
    assert hr_row['Leave'] == 3
    # Engineering has 1 Leave and 1 Absent and 2 Present
    eng_row = summary[summary['Department'] == 'Engineering'].iloc[0]
    assert eng_row['Leave'] == 1
    assert eng_row['Absent'] == 1
```

```
    # Sales: 1 Leave, 1 Present, 1 WorkFromHome => Note WorkFromHome is not counted (only
Present/Absent/Leave)
    sales_row = summary[summary['Department'] == 'Sales'].iloc[0]
    assert sales_row['Leave'] == 1
    assert sales_row['Present'] == 1

def test_empty_and_none_inputs():
    an = LeaveAnalyzer()
    df_empty = pd.DataFrame(columns=["EmployeeID","Department","Date","Attendance"])
    # empty compute monthly
    mr = an.compute_monthly_leave_rate(df_empty)
    assert isinstance(mr, pd.DataFrame)
    assert mr.empty
    # add_onleave_flag on empty
    flagged = an.add_onleave_flag(df_empty)
    assert 'IsOnLeave' in flagged.columns
    assert flagged.empty
    # frequent onleave on empty
    freq = an.frequent_onleave_employees(df_empty, threshold=0)
    assert isinstance(freq, pd.DataFrame)
    assert freq.empty
    # none input handling
    assert an.create_leave_df(None).empty

def test_missing_columns_raise():
    an = LeaveAnalyzer()
    df_bad = pd.DataFrame({'A':[1,2,3]})
    with pytest.raises(KeyError):
        an.compute_monthly_leave_rate(df_bad)
    with pytest.raises(KeyError):
        an.team_leave_summary(df_bad)
    with pytest.raises(KeyError):
        an.add_onleave_flag(pd.DataFrame({'EmployeeID':[1]}))  # missing Attendance
```
